Lucas Sameh

Assignment 2 · COMP454

1a) Algorithm minmax(L)

Input: A list L of n distinct elements drawn from a totally ordered set

Output: max and min

begin

1. Divide L into $\lceil \frac{n}{3} \rceil$ Sublists such that the first $\lceil \frac{n}{3} \rceil - 1$ sublists contain 3 elements each, while the remaining one contains $n \bmod 3$. (If 0 elements it is size 3)

For each Sublist, SL_i, Find it's median3, Let m_i correspond to each median of SL_i where $i, 1 \le i \le \lceil \frac{n}{3} \rceil$

$m_i = \text{median3}(SL_i[1], SL_i[2], SL_i[3])$

2. Partition L into L' where,

$$L' = \{x \in L \mid x \neq m_i, 1 \le i \le \lceil \tfrac{n}{3} \rceil\}$$

3. if $(|L'| = 2)$ then return L'
else minmax(L')

Time Complexity:

1. Finding the median using median3() as the key operation. We find the median for $\lceil \frac{n}{3} \rceil$ sublists in Step 1.

2. We delete $\lceil \frac{n}{3} \rceil$ elements when partitioning L. This takes $\lceil \frac{n}{3} \rceil$ operations. This may or not be required as only median3 is the key operation.

3. $T(|L'|)$ comparisons. Since, L' eliminates $\lceil \frac{n}{3} \rceil$ elements from L then $|L'| = 2\lfloor \frac{n}{3} \rfloor$, so $T(2\lfloor \frac{n}{3} \rfloor)$ Thus we have the recurrence

$$T(n) = 2\lceil \frac{n}{3} \rceil + T(2\lfloor \frac{n}{3} \rfloor)$$

$a = 1$, $b = \frac{3}{2}$, $f(n) = 2\lceil \frac{n}{3} \rceil$

$\log_b a = \log_{\frac{3}{2}} 1 = 0$, $n^0 = n^{\log_{\frac{3}{2}} 1}$

$n^1 = n^{\log_{\frac{3}{2}} 1 + \varepsilon}$, where $\varepsilon = 1$

$f(n) = \Omega(n)$ is true obviously.

Regularity Property: If $af(n/b) \leq c \cdot f(n)$ where $0 < c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$

1. $f(n/3/2) \leq c \cdot f(n)$

$= f(n \cdot \frac{2}{3}) \leq c \cdot f(n)$

$= 2\lceil \frac{\frac{2}{3}n}{3} \rceil \leq c \cdot 2\lceil \frac{n}{3} \rceil$

$= 2\lceil \frac{2}{9}n \rceil \leq c \cdot 2\lceil \frac{n}{3} \rceil$

Let $c > \frac{2}{3}$ and $c \leq 1$

Then regularity property is satisfied.

$\therefore T(n) = \Theta(2\lceil \frac{n}{3} \rceil) = \Theta(n)$

$\Theta(n) \Rightarrow O(n)$

∴ the algorithm finds max and min of L using median3 in $O(n)$ time.

b) Algorithm medianSort(L[lower..upper])

input: A list L[lower...upper] of n distinct elements drawn from a totally ordered set.
output: L sorted in ascending order.

1. if (lower < upper) then
   medianSort(L, lower, $\lfloor \frac{lower + upper}{2} \rfloor$);
   medianSort(L, $\lfloor \frac{lower + upper}{2} \rfloor + 1$, upper);

Merge(L[lower...L[$\frac{lower+upper}{2}$]], L[L[$\frac{lower+upper}{2}$]+1...upper]);

Algorithm Merge(L[lower...mid], L[mid+1...upper])

input: Two lists, L[lower...mid] and L[mid+1...upper] that are sorted.

output: A sorted list C[1...upper] of size of both lists added together.

Begin:

~~// this will cause median3 to return the index of the lowest besides INTMIN (INTMIN is like -∞)~~

~~while (lower ≤ mid and~~

~~median = Median3(L[lower], L[mid+1], INTMIN)~~

~~interest~~

Let a=lower; b=mid+1; c=1

medium = median3(L[a], L[b], INTMIN)
// where INTMIN is the lowest
// possible integer. This causes
// median3 to return the lowest
// between L[a] and L[b].
C[c] = L[median]
if(C[c] contains L[a])
increment a; else increment b;

```
increment c;

while(a≤mid and b≤upper)do
    median=median3(L[a],L[b],L[c-1])
    // since L[c] is already largest
    C[c]=b[median]
    if(C[c]==L[a])
        increment a;
    else increment b;
    increment c;
endwhile;

if(a>mid) then
    copy L[b...upper] into C
else copy L[a...mid] into C

return C
```

Time complexity

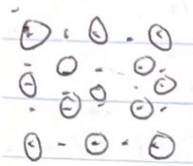
$$T(n)=\begin{cases} T(\lfloor \frac{n}{2} \rfloor)+T(\lceil \frac{n}{2} \rceil)+(n-1), n>1 \\ 0 \qquad\qquad\qquad\qquad\qquad\quad n\leq 1 \end{cases}$$

This is the same as merge sort which is

$\Theta(n\lg n) \Rightarrow O(n\lg n)$

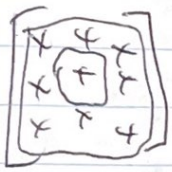
nxn array of distinct R numbers A.
Neighbours of entry A[i,j],
$1 \le i,j \le n$, are the entries

5x5



4n-4

A[i-1, j], A[i+1, j], A[i, j-1],
A[i, j+1].



border

Entry is a min if it's number is less than those of its neighbour.

B = set of all entries on the border of A.

A satisfies: $\exists c, c \notin B, c \in A$,
neighbour(c) $\in B$ and
neighbour(c) = ~~min~~ $\{b \mid b \in B\}$ min(b)$\}$

Find O(n) algorithm that finds a min in A.

Idea: Only search the outside of A (ie. B) for min's. We cannot check every element as it has n^2 elements. That would have 4n-4 operations.
Or check the second ring of numbers

2a) Algorithm FindMin(A)

input: an nxn list of real distinct numbers A

output: A min of A

begin;

B = the border entries of A

1. Find the minimum element in B, called minB, minB = B[k,q] for some k,q

2. For (i=2 to n-1) do
    if (A[i,2] < minB)
        minA = minB
    ~~else if~~
    else if (A[2,i] < minB)
        minA = minB
    ~~end for~~
    else if (A[n-1,i] < minB)
        minA = minB
    ~~End for~~
    elseif (A[i,n-1] < minB)
        minA = minB

This gives us a minA that is atleast less than b, ∀b∈B. minA has some k,q where minA = A[k,q].

3. Check the neighbors of minA and if ~~it is the~~ minA is the smallest return minA.

End Procedure

~~An~~ A min may not always exist in A, and the algorithm cannot find mins past the first two borders. I am not sure how else in O(n) Time Complexity.

1. Finding the minimum element in B will take 4n-4 operations

2. The for loop runs n-2 times and there are 4 comparisons inside, so it takes 4(n-2) = 4n-8 operations.

3. At most it will take 3 operations

$T(n) = 4n-4+4n-8+3$
$= 8n+9 = O(n)$

b) I don't know how for b